

AI FOR IMPACT (AI4I) CHALLENGE 2026

RimaAI

AI-Powered Smart Farming Companion for Zimbabwean Farmers

Track 3 — Development

Field	Details
Project Title	RimaAI — Smart Farming Companion
Track	Development
Team Name	Team RimaAI
Team Members	Lester Rusike (Lead Innovator — Backend, ML & Mobile) · Agnes Goora (Frontend Developer)
Contact	lesterrusike@gmail.com
Live Demo	https://rimai.vercel.app/ — UI demo, Android APK & GitHub source all linked from this one page (Appendix A)
Date	14 July 2026

We believe a small solution that works, is understood by the team, and can be piloted responsibly beats a polished demo that cannot be explained or deployed.

1 Problem Definition & Strategic Alignment

Zimbabwean agriculture employs the majority of the rural population and underpins food security, yet smallholders repeatedly suffer avoidable crop and livestock losses. The defining pattern: preventable or treatable problems compound because farmers lack timely, trusted, local-language information. The clearest evidence is the January disease (theileriosis) outbreak — a tick-borne cattle disease that killed on the order of hundreds of thousands of cattle in Zimbabwe between 2017 and 2021. It is preventable by regular dipping and treatable with available medicine; farmers simply did not know correct schedules or medicines, and extension coverage had collapsed. Information access was the killer, not incurability.

Problem (evidence-led)	RimaAI answer
January disease devastated the national herd; treatable but farmers lacked dipping/medicine information	Livestock advisor + SMS/USSD dipping & vaccination reminders + outbreak alerts
Crop diseases (Maize Lethal Necrosis, blight, fall armyworm) identified too late	Offline photo scanner with instant treatment advice
Erratic rainfall / climate shift leads to wrong planting windows	Localized planting-window forecasts
AGRITEX extension officers are stretched thin — one officer serves hundreds of farmers	AI triage handles first-line questions; officers get a dashboard of flagged cases
Stock theft is rampant, especially for commercial farmers	RimaAI Guard camera-based surveillance module
Low insurance literacy leads to total loss on disaster	Insurance-tips channel + partner linkage
Most agricultural content is English-only, excluding many rural users	Shona / Ndebele / English throughout every channel

Extension services are under-resourced relative to farmer numbers, and smartphone-only apps exclude feature-phone users on costly, intermittent data. RimaAI is therefore offline-first on smartphones and reachable over USSD/SMS/WhatsApp on basic phones, in the farmer's own language.

Strategic alignment

- Zimbabwe National AI Strategy — agriculture is a named priority domain; RimaAI is a deployable AI application plus digital-inclusion by design.
- NDS1 (National Development Strategy 1) — agricultural productivity, food security, rural digital transformation.
- Digital & financial inclusion — the USSD/SMS layer and insurance-literacy content reach farmers that app-only tools miss.

2 Technical Design & Product Logic

A single FastAPI backend serves three farmer segments — smartphone (offline-first Flutter app), feature phone (USSD/SMS/WhatsApp), and commercial (Guard) — over a Supabase/Postgres datastore. Eight routers (scan, triage, forecast, subscriptions, alerts, guard, outbreaks, ussd) expose the full product surface; the USSD router implements the

Africa's Talking callback contract, exercised by the bundled web simulator. Full diagram in docs/architecture.md; full API reference in docs/api.md.

Architecture components

Component	RimaAI implementation
Frontend	Flutter mobile app (offline-first) + USSD/SMS/WhatsApp channels + Next.js web dashboard for AGRITEX officers.
Backend	FastAPI, 8 routers, pluggable messaging (console/Twilio) and triage (rule/LLM) backends.
Database	Supabase-managed Postgres with Row Level Security; full DDL in sample_data/schema.sql.
AI layer	On-device TFLite classifier, LLM triage w/ rule fallback, YOLOv8n detection, weighted-rule risk scoring.
Integration layer	Africa's Talking (USSD), Twilio (SMS/WhatsApp), MSD/satellite weather (roadmap).
Authentication	Supabase Auth; service-role writes for alerts/guard events; farmer-scoped RLS reads.
Monitoring & security	Structured logs, Supabase dashboard, .env-based secrets, Pydantic input validation on every request.

Models and the AI justification

Capability	Model / method	Why AI is warranted
Crop/livestock disease scan	MobileNetV3, quantized TFLite, on-device	Image classification is a genuine CV task — impossible with rules/SQL.
Livestock triage	LLM (Claude) with transparent rule fallback	Mixed Shona/English symptom text needs language understanding; the keyword baseline provably fails on paraphrase.
Guard intrusion detection	YOLOv8n object detection, server-side	Must distinguish person/vehicle from cattle at night — class-aware detection, not a motion-pixel threshold.
Forecast	Time-series over MSD + satellite data (seeded stub today)	Sparse, noisy regional weather data.

AI is deliberately not used for subscriptions, alert scheduling, or outbreak risk aggregation — district risk is a transparent weighted rule ($\sum \text{reporter-trust} \times \text{recency}$), auditable rather than a black box. Spread-prediction ML is a roadmap item, unlocked once real report volume exists.

Disease scans run entirely on-device (target: <256 MB RAM, <100 ms inference on a mid-range Android device); the server /scan endpoint is a fallback only. The app is offline-first end to end: writes (scans, outbreak reports, subscriptions) are optimistic and queued locally, then replayed idempotently on reconnect. Community outbreak reports recompute per-district risk on every submission; crossing the high-risk threshold auto-dispatches a region-targeted SMS alert — crowdsourced intelligence closing the loop back into the alert system. Schema: regions, farmers, subscriptions, alerts, scan_history, guard_events, outbreak_reports, district_risk (full DDL with Row Level Security in sample_data/schema.sql).

3 Deliverables & Roadmap

This submission ships a working FastAPI backend (all eight routers, pluggable messaging and triage backends, seed data, 20 passing pytest tests); a Flutter app (trilingual onboarding, crop scan, livestock triage chat, consent-gated alerts, Guard camera view, outbreak heat map, widget tests); a USSD web simulator; a Supabase schema with Row Level Security; and full documentation (README, architecture, API, dataset statement).

Horizon	Milestone
Through bootcamp (27 Jul–1 Aug)	Train first real MobileNetV3 crop model on a PlantVillage subset; measure/report on-device RAM & latency; wire one live Twilio sandbox for SMS; deploy backend to the ZCHPC CCE and run the full test suite there.
3 months	Field-collect Zimbabwean crop images with AGRITEX in one pilot district; harden offline sync; live USSD shortcode pilot; YOLOv8n Guard model on real night footage.
6 months	MSD weather integration; AGRITEX officer dashboard of flagged triage cases; WhatsApp bot in production; expand crop/livestock classes.
12 months	Outbreak spread-prediction model on accumulated report data; multi-district scale-out; institutional integrations.

Training is modest — transfer learning on MobileNetV3/YOLOv8n fits a single mid-range GPU; serving is CPU-friendly since the phone does most inference. We will use the ZCHPC Controlled Compute Environment to run reproducible training jobs from locked manifests, host the pilot backend and Postgres instance, and execute the automated test suite in CI on every change. Dependencies are pinned: backend/requirements.txt (compiled lockfile) and app/pubspec.lock.

4 Compliance, Risk & Responsible AI

AI advice can be wrong, and wrong advice about animals or crops has real cost. Every scan and triage result carries a disclaimer and an explicit escalation to AGRITEX or a veterinarian — the AI is first-line guidance, never a replacement for an expert. Urgent triage outcomes surface a “contact a vet immediately” message, and all advice content will be reviewed and signed off by AGRITEX/veterinary services before the pilot.

Risk & compliance checklist

Safeguard	Status & evidence
Data minimisation	Implemented — only a phone number (+ optional name/region) is collected; no location tracking beyond a self-selected district.
Consent	Implemented — registration refused server-side without consent_given; trilingual consent checkbox at signup; USSD subscription dial recorded as consent.
Access control	Implemented — Supabase Row Level Security restricts each farmer to their own records (sample_data/schema.sql).
Authentication	Implemented — Supabase Auth for identity; alerts/guard events written only by the backend service role.

Safeguard	Status & evidence
Secrets management	Implemented — no keys in the repository; secrets via environment variables with a committed <code>.env.example</code> .
Encryption	Implemented — Supabase-managed encryption in transit and at rest.
Auditability	Partial — scan/triage history logged; formal audit-log export planned for pilot.
Human oversight	Implemented — every AI output carries an AGRITEX/vet escalation path; urgent cases flagged explicitly.
Misuse risk	Assessed — Guard surveillance limited to opted-in commercial sites; no biometric or facial recognition used.
Bias & fairness	Assessed — trilingual UI; district-level weighting avoids single-language/region skew; PlantVillage-to-Zimbabwe field gap documented in <code>dataset_statement.md</code> .

Data Protection Act [Chapter 11:12] compliance is addressed through the consent, minimisation and access-control rows above.

5 Business Model & Sustainability

Field	Response
Problem	Avoidable crop/livestock losses from delayed, inaccessible agricultural information.
Primary user	Smallholder and commercial farmers, via smartphone, feature phone, or USSD.
Beneficiary	Farming households, AGRITEX extension officers, and — indirectly — national food security.
Customer or payer	Free for smallholders; commercial farmers pay for Guard; institutions (insurers, NGOs, agritech firms) license aggregated intelligence.
Value proposition	Faster, trusted, local-language treatment advice; theft deterrence; early-warning outbreak alerts.
Revenue or funding model	Freemium: free core (scans, triage, forecasts, alerts) + paid Guard (per camera/site) + institutional licensing.
Cost drivers	Hosting/database/storage, SMS/USSD (main variable cost), model training/compute, field data collection, part-time personnel.
Partnerships	AGRITEX (pilot access, content sign-off, field data), a telco/SMS aggregator, ZCHPC (compute).
Pilot market	One district reached through AGRITEX with both smallholder livestock and maize cropping, exercising every module.
Adoption risks	SMS cost sensitivity for farmers; trust-building around AI advice; connectivity gaps in remote wards; AGRITEX/vet sign-off timeline.
Success metrics	Farmers reached per channel; scan/triage usage; alert delivery rate; AGRITEX-reported qualitative impact on early treatment — tracked at 30/60/90 days.

Item	3-month pilot (1 district)	1-year scale (multi-district)
Cloud hosting + database	US\$300–600	US\$3,000–6,000

Item	3-month pilot (1 district)	1-year scale (multi-district)
SMS / USSD (telco)	US\$500–1,500	US\$8,000–20,000
Model training / compute	US\$200–500 (CCE-subsidised)	US\$2,000–5,000
Field data collection (with AGRITEX)	US\$1,000–2,000	US\$6,000–12,000
Personnel (part-time)	US\$3,000–5,000	US\$30,000–50,000
Indicative total	≈ US\$5–9k	≈ US\$50–90k

6 Deployment Plan & User Experience

Field	Response
Deployment environment	Hybrid — cloud-hosted backend/database, offline-first mobile client, static-hosted web dashboard.
Hosting provider or site	Web dashboard/landing on Vercel (live: rimai.vercel.app); backend + Postgres via Supabase, with training and CI test runs in the ZCHPC CCE.
Operator	Team RimaAI through the pilot; transitioning toward AGRITEX co-ownership of content and district rollout post-pilot.
Pilot site	One AGRITEX-selected district with both livestock and maize farming.
Users to onboard	100–300 farmers in the first 90 days, via AGRITEX officer-led induction sessions.
Training and support	In-app trilingual walkthrough, USSD menu help text, AGRITEX officer community demos; support via WhatsApp/SMS and an admin panel for flagged cases.
Monitoring	Structured application logs, Supabase dashboard for DB health, uptime checks on the API, and an incident log for triage/scan errors.
Backup and recovery	Supabase automated daily backups with point-in-time recovery; periodic CSV exports of core tables; version-controlled schema migrations.
Connectivity plan	Offline-first app with local queue and idempotent sync; USSD/SMS as the zero-app-install fallback for feature phones.
Scale pathway	District-by-district expansion via AGRITEX, milestone-gated at 30/60/90 days then 3/6/12 months (Section 3).

User experience

Primary persona: Tendai, 42, a smallholder in a maize- and cattle-farming district, shares a smartphone with family and also owns a basic phone; data budget is small; first language is Shona. User journey: a crop shows symptoms → Tendai opens the app or dials the USSD menu → photo scan or symptom description → instant advice with an urgency flag → if urgent, an explicit prompt to contact AGRITEX/a vet → the outcome is logged locally and syncs to the farmer's history and, for outbreaks, to the district risk map. The screens listed as delivered in Section 3 (trilingual onboarding, scan, triage chat, subscriptions, Guard camera, outbreak map) are working implementations, not mockups. Accessibility: large tap targets, high-contrast UI, Shona/Ndebele/English throughout, and full functionality offline.

Structured usability testing with an AGRITEX-facilitated farmer group is scheduled for the pilot — not yet conducted, logged honestly rather than claimed.

7 Team Capability & Implementation Plan

Name	Role	Skills contributed
Lester Rusike	Lead Innovator — Backend, ML & Mobile	FastAPI backend, Supabase/Postgres schema & RLS, Flutter offline-first app, TFLite/ML pipeline, systems architecture.
Agnes Goora	Frontend Developer	Web dashboard and landing experience (Next.js), UI/UX consistency across the app and web surfaces.
Open role	Agronomy / veterinary content reviewer	Gap — to be filled via the AGRITEX partnership and bootcamp mentor support before pilot content sign-off.

Known gaps: dedicated ML/data-engineering capacity for model training at scale, formal agronomy/veterinary sign-off on advice content, and telco/business-development relationships for SMS/USSD pricing. These are mitigated through the AGRITEX partnership (field data and content review), POTRAZ bootcamp mentor support, and planned part-time recruitment of an agronomist/vet advisor and a business/ops lead once pilot funding is secured (Section 5 budget). Lester Rusike owns maintenance and communication after the challenge.

External support declared

AI coding assistants (Claude Code) were used to accelerate implementation across the backend, app, and documentation. The team reviewed, tested, and can explain every architectural decision and code path; no unexplained generated code is submitted as-is. No third-party datasets, code templates, or partner code were incorporated beyond the open-source libraries declared in requirements.txt and pubspec.yaml.

Period	Focus
0–30 days	Clean codebase, incorporate bootcamp judge feedback, confirm AGRITEX pilot partner, complete the security checklist (Section 4).
31–60 days	Run the pilot with limited users, collect feedback, fix defects, improve onboarding, document measurable outcomes.
61–90 days	Prepare the adoption case, refine the business model, finalise the support plan, update technical documentation and the scale pathway.

A Appendix — Demo Access, Repository & Data Honesty Statement

Everything a judge needs is reachable from one URL: <https://rimai.vercel.app/>. That page is the single hub for this submission and links directly to all three of the following.

Evidence item	Where on rimai.vercel.app
UI demo	The live AGRITEX officer dashboard (/dashboard) — reads real district risk, alerts and Guard events from the deployed backend; no install required.
Android APK	Release build (arm64), linked at /downloads/RimaAI.apk — pre-configured to point at the live backend by default, installable directly on a test device.
GitHub source	Full repository (backend, Flutter app, web dashboard, docs, tests) — linked from the navigation bar and the evaluator-links section on the homepage.

To run locally instead: see README.md. Backend — `pip install -r backend/requirements.txt` → `python -m app.seed` → `uvicorn app.main:app --reload` → `pytest`. App — `flutter create .` && `flutter pub get` && `flutter run`. USSD — open `ussd-simulator/index.html`.

Data honesty statement: disease/livestock classifiers are documented placeholder models in this MVP, with an identical inference interface to the trained model so it is swappable with no application changes; the forecast and Guard modules use seeded/sample data. Full detail and licences are in `docs/dataset_statement.md`. All advice is framed as first-line guidance pending AGRITEX/veterinary sign-off — never a diagnosis of record.